

EDS Migration Strategy	2
Migration Strategy --Experience Fragments	3
Migration Strategy -- Content Pages	14
Migration Strategy -- Forms	26
Migration Strategy -- Content Syndication Pods	38
Migration Strategy -- Reference Component	45
Migration Strategy -- Redirects	51
Migration Strategy -- Metadata	55
Assets Migration	59
Content Management & Localization	62
Multi-Site Manager (MSM)	63
Experience Fragments	71

EDS Migration Strategy

Content Syndication Pods — Migration Strategy

ACS Commons Generic Lists – Migration Strategy

migration of default offers XF before page scraping

friendly urls/ vanity urls migration

migration of users and groups

migration of XF

Forms – rules, constraints etc	documented
xf – forms, site	documented
content syndication pods	done
default offers XF	done
friendly url migration	
content migration	docuemnted
redirects	done
metadata	
Assets/DAM	done
reference Components	done

Migration Strategy --Experience Fragments

Overview

This document defines the migration strategy for TFS Experience Fragments (XFs) from AEM 6.4 to DA. It covers two categories of XFs — Forms XFs and Site XFs — each requiring a different migration approach due to differences in their structure and consistency.

Sections 1–7 cover Forms XF migration — the structural transformation from AEM's variant model to DA's fragment and inheritance model.

Section 8 covers Site XF migration — which requires a different approach due to the absence of a consistent structural pattern across TF Site XFs.

Forms XF Migration

1. The Fundamental Change — No XF or Variants in DA

In AEM, Experience Fragments are a dedicated content type. Each XF holds multiple **variations** — a master (global default) and additional region or country-specific variants — all under one XF node. A custom best-match resolution logic in the form component selects the correct variant at runtime based on the current site context, falling back to master if no specific variant exists.

DA has no Experience Fragment concept and no variation model.

Everything in DA is a document — a fragment is simply a document stored in a designated `/fragments/` folder. There is no container that groups variants together, no selection logic, and no fallback code.

This means every AEM XF variation must become its own **independent DA fragment document**, placed in the folder that corresponds to its scope — global, regional, or country level. The custom best-match resolution logic is retired entirely. DA MSM inheritance takes over this responsibility natively.

2. Structural Transformation — Before and After

AEM Today — Flat Variant Model

All variants for a given XF sit under one XF node. Selection is handled by custom code in the form component.

```
1 /content/experience-fragments/tfforms/fieldsets/standard/marketing-
  opt-in/
2   └─ master          → Global default
3   └─ en-europe       → EMEA region
4   └─ en-north-america → North America region
5   └─ en-ca           → Canada specific
6   └─ en-kr           → Korea specific
7   └─ zh-cn           → China Chinese
8   └─ en-jp           → Japan specific
9
10 Selection: Custom best-match code checks current site context
11           → finds closest variant → falls back to master if none
12 found
13
```

DA After Migration — Distributed Across Hierarchy

Each variant becomes a standalone fragment placed at the correct level of the DA site hierarchy. No custom selection code — DA MSM inheritance handles fallback natively.

```
1 global/en/fragments/forms/standard/marketing-opt-in ← master
2 europe/en/fragments/forms/standard/marketing-opt-in ← en-
  europe
3 north-america/en/fragments/forms/standard/marketing-opt-in ← en-
  north-america
4 north-america/en-ca/fragments/forms/standard/marketing-opt-in ← en-ca
5 ipac/en-kr/fragments/forms/standard/marketing-opt-in ← en-kr
6 greater-china/zh-cn/fragments/forms/standard/marketing-opt-in ← zh-cn
7 japan/en-jp/fragments/forms/standard/marketing-opt-in ← en-jp
8
9 Selection: DA MSM inheritance chain – no custom code required
10 Fragment exists at country level → serve it
11 No country fragment → inherit from regional hub
12 No regional fragment → inherit from global/en (master)
13
```

Placement Decision Rule

Variant scope	DA placement
Global default (master)	global/en/fragments/forms/. ..
Regional variant (e.g. en-europe, en-north-america)	Regional EN hub folder (e.g. europe/en/fragments/forms/. ..)

Country-specific variant (e.g. en-ca, en-kr)	Country site folder (e.g. north-america/en-ca/fragments/forms/...)
Translated variant (e.g. zh-cn, ko-kr)	Translated locale folder (e.g. greater-china/zh-cn/fragments/forms/...)

Only variants that exist in AEM are migrated. Countries with no specific variant rely on DA MSM fallback to serve the regional or global fragment — no empty fragments are created.

3. DA Fragment Folder Structure

Form fragments in DA are organised into two folders mirroring the current AEM structure:

```

1  /fragments/forms/
2  |— standard/      → Standard form field fragments (opt-in,
   | address, contact info etc.)
3  |— divisions/    → Division-specific fragments
4  |   |— cad/
5  |   |— lpd/
6  |   |— mbd/
7  |   |— cmd/
8  |   |— ...
9  |

```

This folder structure exists within each DA site — global, regional hubs, and country sites — so that DA MSM fallback operates consistently across both standard and division fragments.

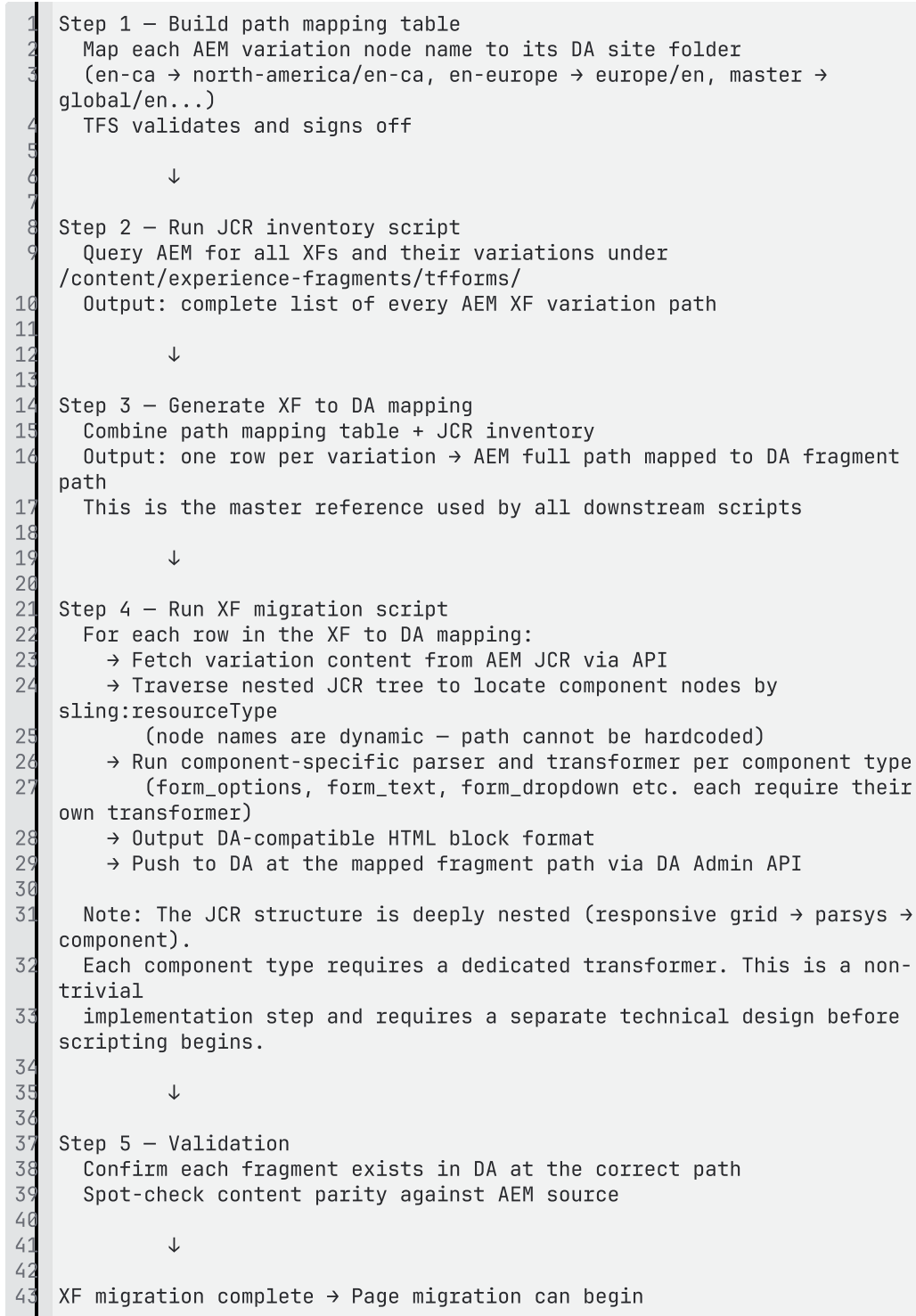
4. Migration Approach

Prerequisites

#	Prerequisite	Detail
1	DA MSM structure set up	All satellite sites configured, repos exist, inheritance chain established. Without this fragments have nowhere to be pushed to

2	Path mapping table completed and validated by TFS	Maps every AEM variation node name to its corresponding DA site folder. Signed off before migration runs
---	---	--

Migration Flow



5. XF to DA Mapping — The Master Reference

The XF to DA mapping document is the single most important artifact in this migration. It is produced by combining the path mapping table with the JCR inventory and serves two purposes:

Purpose 1 — Drives fragment creation in DA The XF migration script reads this mapping to know where to push each fragment.

Purpose 2 — Drives reference replacement during page migration When migrating pages, any AEM XF reference encountered is looked up in this mapping and replaced with the corresponding DA fragment path. Without this mapping, XF references in migrated pages would be broken.

AEM XF Variation Path	DA Fragment Path
.../marketing-opt-in/master	global/en/fragments/forms/standard/marketing-opt-in
.../marketing-opt-in/en-ca	north-america/en-ca/fragments/forms/standard/marketing-opt-in
.../marketing-opt-in/en-europe	europe/en/fragments/forms/standard/marketing-opt-in
.../cad/CAD_Job_Role/master	global/en/fragments/forms/divisions/cad/cad-job-role
.../cad/CAD_Job_Role/en-kr	ipac/en-kr/fragments/forms/divisions/cad/cad-job-role
.../cad/CAD_Job_Role/ko-kr	ipac/ko-kr/fragments/forms/divisions/cad/cad-job-role

6. Best-Match Logic — Retired, Not Replaced

The current AEM best-match resolution logic is a custom code mechanism — it checks the current site context, finds the closest matching XF variant, and falls back to master. This logic is **retired entirely** as part of this migration.

No equivalent code needs to be written in EDS. DA MSM inheritance handles this natively and more reliably:

- A country site with a specific fragment → DA serves that fragment
- A country site with no fragment → DA falls back to the regional hub fragment
- A regional hub with no fragment → DA falls back to `global/en` (master)

The only requirement to make this work correctly is accurate placement of fragments during migration. Each variant must land at the right level of the hierarchy. The XF to DA mapping and TFS sign-off are the controls that ensure this.

7. Migration Timing — Wave 0 Prerequisite

XF fragment migration must complete before any page migration begins. Pages reference form XFs — if fragments are not in DA when pages are migrated, all form references in migrated pages will be broken.

```
1 Wave 0:  DA MSM structure setup
2          XF to DA mapping completed and validated
3          XF migration script runs – all fragments pushed to DA
4
5 Wave 1+: Page migration begins
6          Page migration scripts use XF to DA mapping
7          to replace AEM XF references with DA fragment paths
8
```

8. TF Site XF Migration

8.1 The Problem — No Consistent Pattern

Forms XFs follow a predictable structure — one folder, variations with locale codes, custom best-match resolver. A single migration script can cover all Forms XFs reliably.

TF Site XFs have no consistent pattern. This is the fundamental difference and the primary migration risk.

Observed inconsistencies in the AEM Site XF structure:

- XFs are organised across **Global, North America, Latin America, Europe, Japan, Greater China, India & Pacific** regional folders — but also exist under **Corporate, Homepage, BID,**

Support, Brightcove Experiences, B2B and division folders (BPD, CMD, BPG etc.) with no regional affiliation

- Some XFs sit under `us/en` in AEM but contain content and variations for **entirely different regions** — for example: `/content/experience-fragments/tfsite/us/en/customer-service/europe` This XF lives under `us/en` in AEM but holds European country variations (UK, Germany, France, Spain etc.). The AEM path is structurally misleading.
- Default offers sit under `us/en/site/dm-offers` in AEM but logically belong in global scope — they are not US-specific content
- Variation naming is inconsistent — some XFs use `germany--english`, `france--french` (descriptive country--language names with explicit `countries` and `languages` properties on the node); others use a different convention entirely

Risk: Without upfront cleanup and path alignment, not all Site XFs will be migrated correctly. Gaps are likely if the script runs against an uncurated inventory.

8.2 What Changes in DA

In AEM, Experience Fragments are a separate content type that exists outside the page hierarchy. Authors organise them freely — there are no structural rules enforced by the platform.

In DA, fragments are documents that live inside the site hierarchy — the same hierarchy as pages. This is a fundamental model change. The folder position of a fragment directly determines how DA MSM inheritance resolves it. A fragment placed at `europa/en/fragments/` is inherited by all European country sites that have no country-level override. A fragment placed at `global/en/fragments/` is the top-level fallback for all sites worldwide.

Placement of fragments during migration is therefore critical. A fragment pushed to the wrong folder either breaks inheritance silently or causes the wrong content to be served to the wrong region. The path alignment exercise and the placement rules defined below are the controls that ensure every fragment lands correctly.

8.3 Prerequisite — TFS Cleanup and Path Alignment

Before any migration script runs, TFS must complete the following:

#	Action	Detail
---	--------	--------

1	Confirm production content for migration	Current assessment of Site XF structure is based on content observed during discovery. TFS to confirm that production XF content is in scope for migration and that the XF inventory represents live, active content
2	Remove unused and deprecated XFs	TFS to identify and remove any XF that is no longer referenced by any live page or is no longer needed. Migrating unused content into DA creates noise and ongoing maintenance burden
3	Confirm reorganisation decisions	TFS to confirm where structurally ambiguous XFs should land in DA — specifically XFs under us/en that are not US content (e.g. the europe XF, default offers). Adobe cannot make this call — content ownership sits with TFS

Only after TFS completes this exercise does the migration path alignment and script execution begin.

8.4 Placement Rules — How AEM XFs Map to DA Fragment Paths

Once TFS has curated the inventory, the following rules govern where each XF lands in DA:

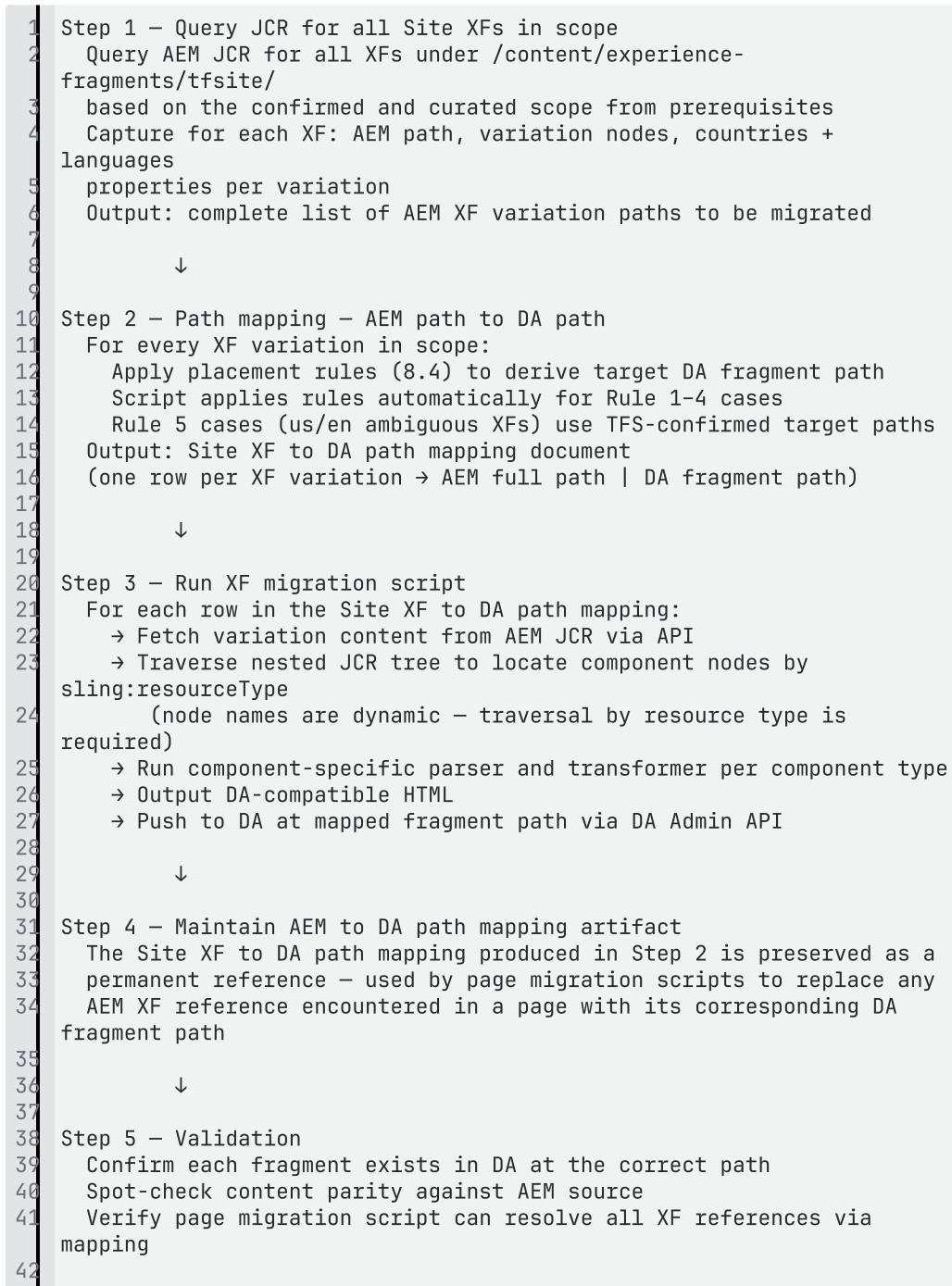
Rule	AEM location	DA target
1	XF under Global folder	global/en/fragments/...
2a	XF under regional folder (Europe, North America, Japan etc.) — master variation	Regional EN hub — e.g. europe/en/fragments/...
2b	XF under regional folder — country variation (with countries + languages properties)	Country DA site — e.g. europe/de-de/fragments/... for Germany German
3	XF outside any regional folder (Corporate, Homepage, BID, Support, division folders etc.)	global/en/fragments/...
4	XF under us/en with US-specific content	north-america/en-us/fragments/...
5	XF under us/en but not US-specific (e.g. default offers, europe XF)	Reorganised per TFS decision — typically global/en/fragments/... or appropriate regional hub

Rule 5 examples:

- /content/experience-fragments/tfsite/us/en/customer-service/europe → TFS to confirm target; expected outcome europe/en/fragments/customer-service/europe with country variants at respective European country DA sites

- `/content/experience-fragments/tfsite/us/en/site/dm-offers` → default offers logically belong at `global/en/fragments/offers/...`

8.5 Migration Flow



8.6 How the Mapping Is Used During Page Migration

The Site XF to DA path mapping serves the same dual purpose as the Forms XF mapping described in Section 5 — it drives fragment creation in DA and drives reference replacement

during page migration. When a page is migrated and an AEM Site XF reference is encountered, the page migration script looks up this mapping and replaces the AEM path with the corresponding DA fragment path.

Migration Strategy -- Content Pages

Overview

This document defines the migration approach for the TFS AEM 6.4 to EDS + DA migration. Given the scale of the estate — approximately 40,000 pages in the master site alone, 45 locales, and a deep MSM inheritance hierarchy — the migration strategy is phased and sequenced around the inheritance model. This document covers the foundation of that strategy: the migration of the global master site.

Migration Waves — Summary

Wave	Scope
Wave 0	DA MSM structure setup · Forms XF migration · XF-to-DA mapping table complete
Wave 1	global/en master site — all active pages
Wave 2	Regional EN hubs — delta pages only (/north-america/en , /europe/en , /ipac/en etc.)
Wave 3	Country EN sites — delta pages only (/en-us , /en-uk , /en-au etc.)
Wave 4	Translated locales — all live pages (de-de , fr-fr , ja-jp , zh-cn etc.)

1. Why Global/EN Must Be Migrated First

DA MSM inheritance operates on **page existence**. If a page exists in a satellite site's directory, DA treats it as a local override. If it does not exist, DA serves the page from the base site. This means the base site — global/en — must be fully populated in DA before any satellite site (regional or country) can inherit from it.

global/en is the originator of the entire inheritance chain. Every regional EN hub, every country EN site, and every translated locale traces its inheritance back to this single master. Migrating any satellite site before global/en is in DA serves no purpose — there is nothing for it to inherit from.

The migration order is therefore non-negotiable:

```
1 | global/en → Regional EN hubs → Country EN sites → Translated  
2 | locales
```

2. The Core Challenge — Global/EN Is Not Publicly Accessible

`global/en` is not a public-facing site. It has no live URLs accessible from the public internet. This means standard importer tooling that works against live URLs cannot be used for this site.

There is also no JCR property on these pages that reliably identifies whether a page is a live master page or a stale/test/structural page. The content owners — TFS — are the only reliable source of truth for which pages are active master content.

3. Prerequisites from TFS

Before migration of `global/en` can begin, the following must be provided by TFS:

#	Requirement	Owner	Notes
1	Page URL list — all active master pages under <code>/content/lifetech/global/en</code> that form the base for all satellite sites	TFS Content team	No JCR query can reliably determine this — TFS content owners must confirm the list
2	QA Publish access — those pages must be accessible via the AEM QA publish instance	TFS Infrastructure	Publish HTML is required (not author) to avoid authoring-specific markup in the output
3	Component inventory — list of all AEM component	TFS or Adobe (via JCR audit)	Determines the scope of transform scripts to be written

	types used across global/en		
--	-----------------------------------	--	--

4. Migration Approach

Scenario A — QA Publish accessible (standard path)

If the QA publish instance is accessible (via TFS VPN or internal network):

```

1 | TFS provides page URL list
2 |   → Fetch rendered HTML from QA publish per page (script running on
   |   VPN)
3 |   → Parse and transform HTML to EDS block format (custom transform
   |   scripts per component type)
4 |   → Push transformed output to DA via DA API
5 |
```

Transform scripts are written once per AEM component type and executed against all pages in the URL list. Once `global/en` is fully pushed to DA, it becomes the live base for all satellite site inheritance.

Form pages: Pages containing forms are detected automatically during migration (via the `cmp-p-form-container` CSS class in the scraped HTML) and handled by a separate form migration script that reads the form section from JCR. See **Forms Migration Strategy** for the full detail.

XF references: When a page is migrated, any AEM Experience Fragment references embedded in that page must be replaced with the corresponding DA fragment paths. This replacement is driven by the XF-to-DA mapping table produced during XF migration. See **XF Migration Strategy** for the full detail. XF migration and the mapping table must therefore be complete before page migration begins.

Scenario B — Pages made publicly accessible (preferred path)

If TFS can temporarily expose the `global/en` QA publish pages via a public URL — even for the duration of the migration — HTML scraping can be used directly against those URLs. This eliminates the need for VPN access or internal publish scripting and accelerates the migration pipeline significantly. This is the preferred approach if it can be arranged.

5. What Happens After Global/EN

Once `global/en` is migrated to DA:

- All pages that regional and country satellite sites **cleanly inherit** from `global/en` require no further migration — DA serves them from the base automatically
- Only **delta pages** — pages where a regional or country site has broken MSM inheritance and holds its own content — require individual migration per locale
- A delta identification script (Adobe to build, run against TFS JCR) will determine the exact volume of delta pages per locale before regional and country migration begins

The delta identification script is the critical prerequisite for all migration work beyond Wave 1.

6. Regional EN Hubs Migration

6.1 The Approach

Regional EN hubs (`/north-america/en` , `/europe/en` , `/greater-china/en` , `/ipac/en` , `/latin-america/en`) are not public-facing sites — they exist purely as inheritance intermediaries between `global/en` and the country EN sites beneath them. Not all pages in these hubs are unique regional content — the majority inherit cleanly from `global/en` . Only the delta pages — pages with regional-specific content or broken inheritance — need to be migrated to DA for each regional hub.

The delta identification script determines exactly which pages those are.

6.2 Delta Identification Script

Adobe will build a JCR script that runs against each regional hub path (e.g. `/content/lifetech/north-america/en`). For each page found under that path, the script checks for the following three conditions:

Condition	What it means	JCR signal	DA outcome
Page-level inheritance broken	The entire page has been detached from <code>global/en</code> — full local override	<code>jcr:content/cq:LiveRelationship</code> node present with <code>cq:isCancelled = true</code> or	New page created in DA for regional hub

		<code>cq:isSuspended = true</code>	
Component-level inheritance broken	Page inherits from <code>global/en</code> but one or more components have been locally overridden	One or more child component nodes under <code>jcr:content</code> have <code>cq:isCancelled = true</code>	New page created in DA for regional hub — DA has no component-level inheritance; the entire page becomes a regional override
No inheritance property	Page was created locally within the regional site — never rolled out from <code>global/en</code>	No <code>cq:LiveRelationship</code> mixin present on <code>jcr:content</code> at all	New page created in DA for regional hub

Important — component-level inheritance: In AEM, a page with component-level broken inheritance is still partially managed by `global/en` — the inherited components automatically receive updates from the master. In DA this granularity does not exist. The entire page becomes a full regional override and will no longer automatically receive updates from `global/en`. Any future global changes to that page must be manually synced by the regional team. TFS has been made aware of this trade-off as part of the DA MSM model.

Any page that meets one or more of these conditions is a delta page for that regional hub. The script outputs a flat list of page paths.

6.3 TFS Review Gate

The script output must be reviewed and approved by the TFS content team before migration runs. The delta list will include pages that are stale, no longer needed, or candidates for

retirement — the script identifies all technical delta pages but cannot determine business relevance. Only TFS content owners can make that call.

```
1 Script output (delta page list)
2   → TFS content team reviews
3   → Removes stale, test, or retired pages
4   → Signs off on final list
5   → Migration pipeline runs against approved list only
6
```

This review gate applies to every locale wave — regional hubs, country EN sites, and translated locales.

6.4 Migration Pipeline

The approved delta page list feeds into the migration pipeline:

```
1 For each approved delta page:
2   → Fetch rendered HTML from QA publish (VPN or internal access)
3   → Parse and transform HTML to EDS block format
4   → Push to DA under the regional hub satellite site path
5
```

Pages not in the approved list require no migration — DA serves them from `global/en` automatically.

6.5 Scale Consideration

The volume of delta pages per regional hub directly determines the migration effort for that hub. Until the delta identification script runs, this volume is unknown. The script output should be reviewed before migration begins for each regional hub — it sets the scope and effort estimate for that wave.

7. Country EN Sites Migration

7.1 The Approach

Country EN sites (`/north-america/en-us` , `/europe/en-uk` , `/ipac/en-au` etc.) sit one level below the regional EN hubs in the inheritance chain. The same delta identification approach applies — only pages that differ from what they would naturally inherit need to be migrated to DA. All other pages are served automatically through the inheritance chain.

7.2 Delta Identification Script

The same JCR script used for regional hubs runs against each country EN site path. The three conditions are identical:

Condition	DA outcome
-----------	------------

Page-level inheritance broken	New page created in DA for that country site
Component-level inheritance broken	New page created in DA for that country site — full page override, same trade-off as regional hubs
No inheritance property — locally created page	New page created in DA for that country site

The script output is reviewed and approved by the TFS content team before migration runs — the same review gate defined for regional hubs applies here.

7.3 Three-Level Inheritance Chain

Country EN sites sit at the third level of the inheritance chain — `global/en → regional/en → country/en`. When a delta page is identified at country level, it may be overriding a page that exists in either the regional hub or `global/en`. The HTML fetched from publish always reflects the correct final rendered state of the page — AEM resolves the full inheritance chain at render time. The migration pipeline does not need to trace the chain; it works against the rendered HTML output.

7.4 Migration Pipeline — Two Paths

Country EN sites are publicly accessible, which provides an additional option not available for regional hubs:

Path A — Public URL (preferred)

If the delta pages are accessible via live public URL, HTML scraping runs directly against those URLs. No VPN or internal publish access required. This is the simpler and faster path for country sites.

```

1 Delta page list approved by TFS
2   → HTML scraping runs against live public URLs
3   → Transform scripts process HTML
4   → Output pushed to DA under country site path
5
```

Path B — QA Publish via VPN

If any delta pages are not publicly accessible:

```

1 Delta page list approved by TFS
2   → Fetch rendered HTML from QA publish (VPN or internal access)
3   → Parse and transform HTML to EDS block format
4   → Push to DA under country site path
```

Pages not in the approved delta list require no migration — DA serves them through the inheritance chain automatically.

8. Translated Locales Migration

8.1 Why All Pages Must Be Migrated

Translated locales (`de-de` , `fr-fr` , `ja-jp` , `zh-cn` , `ko-kr` etc.) follow a different pattern from regional hubs and country EN sites. For these locales, **all live pages must be migrated to DA** — not just delta pages.

The reason is the DA MSM inheritance model. If a page does not exist in a satellite site, DA falls back and serves the page from the parent — which for a translated locale would be the EN counterpart. A missing page in `de-de` would serve English content to German users. This is incorrect.

Translated text is local content by definition — it can never be inherited from an EN parent. Every page that carries translated content must therefore exist as an explicit override page in DA. There are no clean inheritance pages in a translated locale.

8.2 Page List — Sitemap Based

Since all translated locales are publicly accessible, the sitemap of each locale is the definitive list of pages to migrate. No JCR delta script is needed and no TFS content review of the page list is required — the live sitemap already represents the confirmed production content for that locale.

8.3 Migration Pipeline

```

1 | Crawl sitemap of translated locale (e.g. de-de)
2 |   → Every page in sitemap is migrated to DA
3 |   → Fetch rendered HTML from live public URL (no VPN required)
4 |   → Parse and transform HTML to EDS block format
5 |   → Push to DA as override page under translated locale path
6 |
```

This pipeline applies to all 13 translated locales: `fr-fr` , `de-de` , `es-es` , `ru-ru` , `zh-cn` , `zt-hk` , `ja-jp` , `ko-kr` , `zt-tw` , `es-ar` , `es-cl` , `es-mx` , `pt-br` .

9. Migration Scenarios — Existing Page Behaviour

This section defines exactly how existing AEM pages behave through migration for each inheritance state. It covers what changes, what stays the same, and what authors will

experience in DA after migration. These scenarios apply across all site levels — regional hubs, country EN sites, and translated locales.

Scenario 1 — Fully Inherited Page

AEM state today

- Page exists in a regional or country site as a live copy of `global/en`
- No inheritance broken at page level or component level
- Author opens the page in AEM — all components are locked and greyed out, nothing is editable
- Any change made in `global/en` automatically rolls out to this page

Migration action

- This page is not migrated for the satellite site
- `global/en` version of the page exists in DA as the base
- No page file is created for this path in the satellite site

DA outcome

- DA detects no page file exists in the satellite site for this path
- DA automatically serves the `global/en` version to the satellite site
- Page is live and rendering correctly for end users — zero migration effort required

Author experience in DA

- Author opens the satellite site folder in DA — this page is not visible there
- This is expected and correct — the page is served from `global/en`
- To make a site-specific change, the author creates a local copy in DA (equivalent of cancelling inheritance in AEM) and edits from there

What changes vs today

	AEM	DA
Page visible in site folder	Yes — locked	Not visible — served from base
Editable by author	No — all locked	Only after creating a local copy
Global updates	Auto rollout	Auto served from base

Scenario 2 — Page-Level Inheritance Broken

AEM state today

- Page exists in a regional or country site
- Entire page has been detached from `global/en` — live copy cancelled at page level
- Author opens the page in AEM — all components are fully editable, nothing is locked
- Changes in `global/en` do not reach this page — it is fully independent

Migration action

- Delta identification script flags this page
- TFS content team reviews and approves it for migration
- Full page HTML is fetched from QA publish
- Page is transformed and pushed to DA as an override page in the satellite site

DA outcome

- Page exists as a file in the satellite site's DA folder
- DA serves the satellite's version — `global/en` version is not used for this page
- Page content is identical to what exists in AEM today

Author experience in DA

- Author opens the satellite site folder in DA — this page is visible and fully editable
- All content is editable — no locks, no restrictions
- Behaviour is identical to what they have in AEM today

What changes vs today

	AEM	DA
Page visible in site folder	Yes — fully editable	Yes — fully editable
Editable by author	Yes — full control	Yes — full control
Global updates	Not received — inheritance cancelled	Not received — full local override

No behaviour change for authors. This is the lowest-risk migration scenario.

Scenario 3 — Component-Level Inheritance Broken

AEM state today

- Page exists in a regional or country site as a live copy of `global/en`
- Page-level inheritance is intact — the page is still a live copy
- One or more specific components have been locally overridden at component level
- Author opens the page in AEM — mixed state:
 - Locked components (greyed out) — still inheriting from `global/en`, auto-updated on rollout
 - Unlocked components (editable) — locally overridden, not affected by `global/en` rollouts
- The page is partially managed by `global/en` and partially owned by the regional or country team

Migration action

- Delta identification script flags this page (component-level `cq:isCancelled = true` detected)
- TFS content team reviews and approves it for migration
- Full page HTML is fetched from QA publish — this HTML contains both inherited and overridden components rendered as one complete page
- Entire page is transformed and pushed to DA as a full override page in the satellite site

DA outcome

- Page exists as a file in the satellite site's DA folder
- DA has no concept of component-level inheritance — the entire page is now a full local override
- All content on the page — previously locked and unlocked — is owned by the satellite site

Author experience in DA

- Author opens the satellite site folder in DA — this page is visible and fully editable
- All components on the page are editable — including what was locked in AEM
- There are no locked or greyed out components — full ownership of the entire page

What changes vs today

	AEM	DA
Page visible in site folder	Yes — mixed editable/locked	Yes — fully editable
Previously locked components	Not editable — inherited from global	Fully editable — owned locally
Global updates to locked components	Received automatically on rollout	Not received automatically
Global updates to overridden components	Never received	Not received
Author control over global updates	None — changes arrive automatically	Full control — author manually syncs when ready

How authors receive global updates in DA

In AEM, changes to locked components arrived automatically when `global/en` rolled out. In DA, the automatic sync is replaced by a deliberate author-controlled sync. When `global/en` updates content, the satellite author manually triggers a sync against the base. DA provides two sync modes:

- **Merge** — brings in base changes while attempting to preserve local overrides. Recommended for pages where local customisations must be retained
- **Override** — fully replaces the satellite page with the base content. Used when the local version needs to be completely refreshed from global

Important change for content governance: Regional and country authoring teams must be notified when `global/en` makes changes to pages they have locally overridden, so they can decide whether and when to sync. This requires a communication process between the global content team and regional teams — a process that did not exist in AEM because rollouts were automatic.

Migration Strategy -- Forms

Overview

This document defines the migration strategy for TFS forms from AEM 6.4 to DA/EDS. TFS forms are built using a suite of AEM core form components with custom extensions — a complex dialog-driven authoring model with 11 action types, XF-based fieldset composition, a custom rule editor for show/hide logic, and server-side constraint aggregation.

In DA/EDS, forms are represented as a sequence of block tables in a DA document, authored through two DA Library Panel plugins and rendered client-side by a single orchestrator block script (`tfs-form.js`).

The migration strategy is a **hybrid approach**: non-form page content is migrated by scraping the page HTML; form sections are migrated from AEM JCR. Both are combined into a single DA document.

1. Why a Hybrid Approach

TFS has large inventory of pages. It is not known upfront which pages contain forms — forms can appear on any page across the site hierarchy. A blanket JCR-only migration for all pages is not practical because:

- Scraping the page HTML captures the fully rendered page with all CSS, styles, and layout intact — this is critical for non-form content (hero, text, images, product sections)
- JCR is required for the form section because the rendered HTML does not contain the structured configuration data needed to reconstruct the form in DA

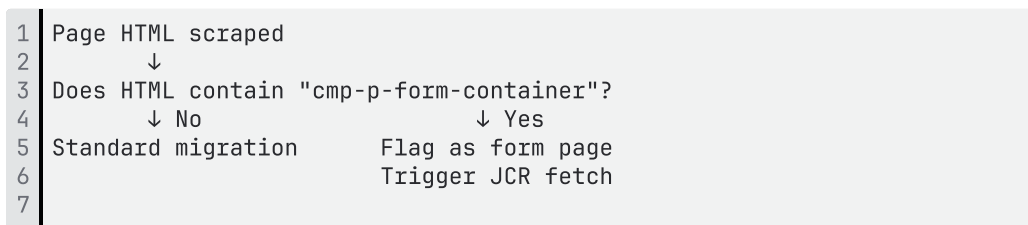
The hybrid approach combines both:

Content	Migration source	Reason
Non-form content (hero, text, images, product sections)	HTML scraping	CSS and layout preserved; no structured config needed

Form section	AEM JCR	Form container config, XF paths, rules, constraints — none available from HTML
--------------	---------	---

2. Form Detection — Identifying Pages with Forms

During page scraping, the migration script scans each page's HTML for the presence of the CSS class `cmp-p-form-container`. This class is rendered by every AEM form container and is a reliable indicator that the page contains a form.



This detection is automatic — no manual tagging or separate form page inventory required.

3. Prerequisites

Before form migration can begin, the following must be in place:

#	Prerequisite	Detail
1	DA MSM structure set up	All satellite sites configured, repos exist, inheritance chain established
2	Forms XF migration complete	All AEM form XF variations migrated to DA as fragment documents at correct hierarchy levels
3	XF to DA fragment mapping table complete	Maps every AEM XF variation path to its corresponding DA fragment path. This table is actively used by the form migration script to resolve every XF inclusion it encounters. Without it, form fragment references cannot be written

XF migration and the mapping table are hard prerequisites — form migration cannot run until both are in place.

4. Migration Approach — Step by Step

Step 1 — Scrape Page HTML

Script scrapes the page HTML. This captures all non-form content — hero sections, text components, product images, promotional content — with full CSS and layout intact.

Step 2 — Detect Form Presence

Script scans scraped HTML for `cmp-p-form-container` class.

- **Not found** → standard page migration, no further action
- **Found** → proceed to Step 3

Step 3 — Strip Form Section from Scraped HTML

Script locates the `cmp-p-form-container` div in the scraped HTML and removes it along with all its rendered children. The position in the document is noted — form blocks will be injected here.

The remaining scraped HTML (hero, text, product content) is preserved as-is for the DA document.

Step 4 — Fetch Form Page JCR

Script calls the AEM JCR API for the same page:

```
1 GET /content/lifetech/<site>/<locale>/...<page-path>.infinity.json
2
```

This returns the complete JCR tree including all form component nodes and their properties.

Step 5 — Run Form Migration Script

This is the core of the form migration. The script traverses the JCR tree and produces DA block tables. See Section 5 for the full detail of what the script does.

Step 6 — Assemble DA Document

Script combines:

- Scraped HTML (non-form content) — from Step 3
- DA form block tables (form section) — from Step 5

All form blocks are placed in a single DA section (no section dividers within the form section — required by `tfs-form.js`).

Step 7 — Push to DA

Assembled HTML document is pushed to DA at the mapped DA path:

5. What the Form Migration Script Does

This section covers the script in detail. The script reads the JCR form container node and produces a sequence of DA block tables — one `tfs-form` container block, one block per field, and one `tfs-form-rules` block if rules exist.

5.1 Locate the Form Container

Script traverses the JCR tree looking for the node with:

```
1 | sling:resourceType = "tfsite/components/form/container"  
2 |
```

Node names in JCR are dynamic and cannot be hardcoded. Traversal by `sling:resourceType` is required.

5.2 Build the `tfs-form` Container Block

From the form container node, extract configuration properties and map to the DA block schema:

JCR property	DA block key	Notes
<code>customActionType</code>	<code>action-types</code>	Comma-separated if multiple (e.g. <code>1001,1006</code>)
<code>eloquaFormName</code>	<code>eloqua-form-name</code>	Present if action type 1001 selected
<code>eloquaInstance</code>	<code>eloqua-instance</code>	Present if action type 1001 selected
<code>division</code>	<code>division</code>	Division code (lpd, cad etc.)
<code>isMqo</code>	<code>mqo-form</code>	Boolean
<code>nonLSGType</code>	<code>nonlsg-type</code>	lead / case / quote / bulk quote

<code>uniqueFormKey</code>	<code>email-key</code>	For email action types 1006/1007/1011
<code>redirect</code>	<code>redirect</code>	AEM redirect path → mapped to DA equivalent
<code>multistep</code>	<code>multistep</code>	Only if multi-step form

5.3 Iterate Field Nodes and Build Field Blocks

Script iterates through all child nodes of the form container in order. Each node's `sling:resourceType` determines which transformer runs:

<code>sling:resourceType</code>	DA block produced	Transformer
<code>tfsite/components/form/input</code>	<code>tfs-form-input</code>	Input transformer
<code>tfsite/components/form/textarea</code>	<code>tfs-form-input</code> (type: textarea)	Input transformer
<code>tfsite/components/form/options</code>	<code>tfs-form-options</code>	Options transformer
<code>tfsite/components/form/button</code>	<code>tfs-form-button</code>	Button transformer
<code>tfsite/components/form/xfinclusion</code>	<code>tfs-form-fragment</code>	XF resolver (see 5.4)
<code>tfsite/components/form/recaptcha</code>	<code>tfs-form-recaptcha</code>	Recaptcha transformer
<code>tfsite/components/form/hidden</code>	<code>tfs-form-hidden</code>	Hidden transformer
<code>tfsite/components/form/upload</code>	<code>tfs-form-upload</code>	Upload transformer

Input transformer extracts from field node:

JCR property	DA block key
<code>jcr:title</code>	<code>label</code>
<code>name</code>	<code>name</code>
<code>type</code>	<code>type</code>
<code>constraints/required</code>	<code>required</code>
<code>constraints/requiredMessage</code>	<code>required-message</code>
<code>constraints/confConstraintMessage</code>	<code>constraint-message</code>
<code>constraints/minlength</code>	<code>minlength</code>
<code>constraints/maxlength</code>	<code>maxlength</code>
<code>placeholder</code>	<code>placeholder</code>
<code>hideonload</code>	Note: field hidden initially — rule handles visibility

Options transformer extracts from field node:

JCR property	DA block key
<code>jcr:title</code>	<code>label</code>
<code>name</code>	<code>name</code>
<code>type</code> (checkbox/radio/select)	<code>type</code>
<code>source</code>	<code>source</code>
<code>items/item*/text</code> + <code>items/item*/value</code>	<code>options</code> (pipe-delimited: <code>Display,value Display,value</code> <code>)</code>
<code>constraints/required</code>	<code>required</code>
<code>constraints/requiredMessage</code>	<code>required-message</code>

5.4 XF Inclusion Resolution — The Critical Step

This is the most complex part of the form migration script.

AEM form XF inclusions store only a base XF path in JCR — no variant information:

```
1 fragmentPath: /content/experience-  
2 fragments/tfforms/fieldsets/standard/contactInformation
```

At AEM runtime, a custom best-match resolver takes this base path and the current site context (locale, region) and resolves it to the correct variation. The migration script must replicate this logic.

Two-step resolution:

```
1 Step 1 – Apply best-match logic  
2   Input: base XF path + site context of page being migrated (e.g.  
3   ipac/en-in)  
4   Logic:  
5     → Does a variation exist for this country locale? → use it  
6     → No country match → check region match  
7     → No region match → fall back to master  
8   Output: resolved AEM XF variation path  
9   e.g. /content/experience-  
10  fragments/tfforms/fieldsets/standard/contactInformation/master  
11  
12  ↓  
13 Step 2 – Look up XF to DA mapping table  
14   Input: resolved AEM variation path  
15   Output: DA fragment path  
16   e.g. global/en/fragments/forms/standard/contact-information  
17  
18   If resolved path not found in mapping table → fall back to master  
19   fragment path
```

Output — tfs-form-fragment block:

```
1 | tfs-form-fragment |  
2 | path              | global/en/fragments/forms/standard/contact-  
3 | information |
```

Why this step is non-trivial:

- Same form JCR produces different DA block configurations depending on which locale is being migrated
- The best-match resolver must be re-implemented in the migration script — it is currently AEM server-side logic
- Every form page being migrated requires the resolver to run for each XF inclusion it contains
- The XF to DA mapping table must be fully populated before this step can execute

5.5 Rule Editor Migration

This is the second area of complexity in the form migration script.

In AEM, show/hide rules are stored as `rules` child nodes on individual field components in JCR — one rules node per field that has a rule. In EDS, all rules for a form are consolidated into a single `tfs-form-rules` block.

Script flow for rules:

```
1 1. Scan all field nodes in the form container
2 2. Build id→name lookup map: for every field node, map its "id"
  property to its "name" property
3 3. For each field node that has a "rules" child node:
4   a. Extract: type (show/hide), check (any/all), target (field ID),
    entries
5   b. Resolve target field ID to field name using the lookup map
6   c. For each entry: build condition string → {controlName}~
   {condition}~{value}
7   d. Write numbered rows to tfs-form-rules block
8 4. Append tfs-form-rules block to DA document (same section as tfs-
  form)
9
```

AEM to EDS rule property mapping:

AEM JCR (<code>rules</code> node)	EDS block key	Notes
<code>type</code>	<code>{n}-action</code>	show → show, hide → hide
<code>check</code>	<code>{n}-logic</code>	any → any, all → all
<code>target</code> (field ID)	<code>{n}-target</code> (field name)	Resolved via id→name lookup map
<code>entries/item*/controlName + condition + search</code>	<code>{n}-cond-{m}</code>	Format: <code>{field}~{operator}~{value}</code>

Example — `C_Others` field rule from our sample form:

AEM JCR:

```
1 "rules": {
2   "type": "show",
3   "target": "form-input-field-1958684667",
4   "check": "any",
5   "entries": {
```

```

6      "item0": {
7          "controlName": "C_TypesofSamples",
8          "condition": "contains",
9          "search": "0thers"
10     }
11 }
12 }
13

```

EDS `tfs-form-rules` block output:

```

1 | tfs-form-rules |
2 | 1-target       | C_Others
3 | 1-action       | show
4 | 1-logic        | any
5 | 1-cond-1       | C_TypesofSamples~contains~0thers
6

```

6. Before and After — Form Structure

This section shows concretely how the form transforms from AEM to DA.

AEM — Current State

The form lives as JCR nodes under the page content tree. The form container holds all configuration as node properties. Fields are child nodes, each with their own dialog properties. XF inclusions reference fragment paths. Rules are stored per-field as JCR child nodes and delivered to the browser as an encrypted payload.

```

1 Page JCR
2   └─ root
3       └─ container
4           └─ container [sling:resourceType:
tfsite/components/form/container]
5               eloquaFormName:
LPD_CTT_PR_FM_0423_Protectingsampleintegrity_APAC
6               eloquaInstance: 642
7               customActionType: 1001
8               division: lpd
9               uniqueFormKey: 2036255305
10
11           └─ xfinclusion_1 [fragmentPath: ../contactInformation]
12               └─ input_1 [name: C_SerialNumber, type: text]
13               └─ xfinclusion_2 [fragmentPath: ../long-form-address-
information1]
14                   └─ xfinclusion_3 [fragmentPath:
../LPD_PrimaryAreaOfResearch]
15                       └─ xfinclusion_4 [fragmentPath: ../LPD_Industry]
16                           └─ options_1 [name: C_TypesofSamples, type:
checkbox, 6 items]
17                               └─ input_2 [name: C_Others, hideonload: true]
18                                   └─ rules [show when C_TypesofSamples contains
0thers]
19                           └─ options_2 [name: C_NumberofCryotubes, type:
checkbox, 4 items]
20                               └─ input_3 [name: C_CommentsRemarks, type: text]
21                                   └─ xfinclusion_5 [fragmentPath: ../Copy of LPD_OptIn]
22                                   └─ recaptcha
23                                   └─ button [type: submit]

```

```

24
25 Selection: AEM best-match logic resolves XF inclusions at runtime
26 Rules: Encrypted data-showhide attribute on <form> element, decrypted
    by showhide.js
27

```

DA — After Migration

The form is a sequence of block tables in a DA document. All blocks in the same section. No server-side logic — `tfs-form.js` reads and renders everything client-side.

```

1 DA Document Section
2 |
3 |— [tfs-form block]
4 |   action-types:      1001
5 |   eloqua-form-name:
LPD_CTT_PR_FM_0423_Protectingsampleintegrity_APAC
6 |   eloqua-instance:   642
7 |   division:          lpd
8 |   mqo-form:          true
9 |   nonlsg-type:       lead
10 |  email-key:          2036255305
11 |
12 |— [tfs-form-fragment block]
13 |   path: global/en/fragments/forms/standard/contact-information
14 |
15 |— [tfs-form-input block]
16 |   label: ULT Serial Number | name: C_SerialNumber | type: text |
required: true
17 |
18 |— [tfs-form-fragment block]
19 |   path: global/en/fragments/forms/divisions/lpd/long-form-address
20 |
21 |— [tfs-form-fragment block]
22 |   path: global/en/fragments/forms/divisions/lpd/lpd-primary-area-
of-research
23 |
24 |— [tfs-form-fragment block]
25 |   path: global/en/fragments/forms/divisions/lpd/lpd-industry
26 |
27 |— [tfs-form-options block]
28 |   label: 1. Which are the types of samples... | name:
C_TypesofSamples
29 |   type: checkbox
30 |   options: Animal cell,Animal cell|Blood,Blood|DNA/RNA,DNA/RNA|...
31 |   required: true
32 |
33 |— [tfs-form-input block]
34 |   label: Others | name: C_Others | type: text | required: true
35 |
36 |— [tfs-form-options block]
37 |   label: 2. Average cryotubes per month | name:
C_NumberofCryotubes
38 |   type: checkbox
39 |   options: Less than 100,Less than 100|101-500,101-500|...
40 |
41 |— [tfs-form-input block]
42 |   label: Comments/Remarks | name: C_CommentsRemarks | type: text
43 |
44 |— [tfs-form-fragment block]
45 |   path: global/en/fragments/forms/divisions/lpd/lpd-opt-in
46 |
47 |— [tfs-form-recaptcha block]
48 |
49 |— [tfs-form-button block]

```

```

50 |     label: Submit | type: submit
51 |
52 | [tfs-form-rules block]
53 |   1-target: C_Others
54 |   1-action: show
55 |   1-logic: any
56 |   1-cond-1: C_TypesofSamples~contains~Others
57 |
58 | Selection: DA MSM inheritance serves correct fragment per locale – no
   | custom logic
59 | Rules: tfs-form-rules block read by tfs-form.js, removed from DOM
   | before render
60 |

```

What Changes — Summary

Dimension	AEM	DA/EDS
Form config storage	JCR node properties	tfs-form block table
Field authoring	Per-field touch-UI dialogs	DA block table rows
XF inclusion	Runtime best-match resolution (custom code)	tfs-form-fragment block with resolved DA fragment path
Show/hide rules	Per-field JCR nodes, encrypted DOM attribute	Single tfs-form-rules block, in-memory evaluation
Constraint delivery	Server-side aggregation, encrypted payload	Field block rows, HTML5 native + lightweight JS
Rendering	Server-side HTL + Sling Models	Client-side tfs-form.js orchestrator

7. Migration Script — Capability Summary

Capability	Approach	Notes
Form detection	Scan scraped HTML for cmp-p-form-container	Automatic — no manual page list needed

Form container config	JCR → <code>tfs-form</code> block	Action types, Eloqua, division, MQO, redirect
Field blocks	JCR field transformer per <code>sling:resourceType</code>	One transformer per component type
Field constraints	JCR <code>constraints</code> node → field block rows	Required, messages, min/max length
XF inclusions	Best-match resolver + XF to DA mapping table lookup	Most complex step — locale-aware resolution
Show/hide rules	JCR <code>rules</code> nodes → <code>tfs-form-rules</code> block	ID→name resolution via field lookup map
Non-form page content	HTML scraping	CSS and layout preserved
DA document assembly	Scraped HTML (non-form) + JCR block tables (form)	Combined into single DA document
DA push	DA Admin API PUT	Per page, at mapped DA path

8. Migration Timing

Form migration follows the same Wave 0 dependency as XF migration:

```

1 Wave 0: DA MSM structure setup
2       Forms XF migration complete
3       XF to DA fragment mapping table complete and validated
4
5 Wave 1+: Page migration begins
6       Every page scraped → form detection runs automatically
7       Form pages: JCR fetch + form migration script
8       Non-form pages: standard scraping pipeline
9

```

Migration Strategy -- Content Syndication Pods

Overview

This document defines the migration strategy for Content Syndication Pods from AEM 6.4 to EDS. Content Syndication Pods are HTML fragments authored in AEM and consumed by the TFS commerce/PDP system to display product-specific marketing content on product pages. The migration moves all pod content into DA, retires the AEM servlet-based delivery mechanism, and replaces it with the `aem-embed.js` Web Component.

It runs **in step with the page migration waves**. When a locale is being migrated, its syndication pods are migrated at the same time.

1. What Are Content Syndication Pods

How They Work Today

The TFS commerce system (PDP/product pages) is a separate platform. To display product-specific content on product pages, it calls out to AEM at runtime and injects the response as an HTML fragment directly into the page.

A single product (identified by a SKU) can have multiple pods — each pod is a distinct section of content on the product page.

The content for each pod is authored as a standalone AEM page using a specialized Content Snippet template. On publish, this page renders as a raw HTML fragment — no page shell, no navigation, no CSS — just the content HTML that the commerce system injects into the product page.

How the Commerce System Requests a Pod

The commerce system calls AEM using a structured URL:

```
1 | /{locale}/product-family-syndicated-  
2 | content.pfpsnippet.html/sku/{SKU}/{POD-NUMBER}
```

The `.pfpsnippet` selector triggers a dedicated AEM servlet (`ProductFamilySyndicationServlet`) which:

1. Parses the SKU and pod number from the URL
2. Constructs the corresponding tag identifier
3. Searches the JCR for a Content Snippet page tagged with that identifier

4. Returns the rendered HTML of that page

Where Pods Live in AEM

Content Snippet pod pages exist across the full site hierarchy — at global, regional, and country levels — mirroring the same structure as regular site pages:

```
1 /content/lifetech/global/en/syndicated-content/snippets/...
2 /content/lifetech/north-america/en/syndicated-content/snippets/...
3 /content/lifetech/north-america/en-us/syndicated-content/snippets/...
4 /content/lifetech/europe/en/syndicated-content/snippets/...
5 /content/lifetech/europe/de-de/syndicated-content/snippets/...
6 /content/lifetech/ipac/en/syndicated-content/snippets/...
7 /content/lifetech/ipac/en-in/syndicated-content/snippets/...
8 /content/lifetech/japan/ja-jp/syndicated-content/snippets/...
9 /content/lifetech/greater-china/zh-cn/syndicated-content/snippets/...
10
```

Each pod page is tagged with a `cq:tag` that identifies which SKU and pod number it serves. The tag namespace is either `pfp:` (Product Family Page) or `pdp:` (Product Detail Page) depending on the type of product page.

2. What Changes in DA

Path Replaces Tags

In AEM, the connection between a URL request and a content page is made through tag taxonomy. The servlet builds a tag ID, searches JCR, and finds the right page — regardless of where that page sits in the folder structure.

In DA, there are no tags and no servlet. **The path where the document is saved IS the identifier.** The commerce system constructs the URL directly from the locale, namespace, SKU, and pod number — no lookup, no tag search.

Commerce System Uses `aem-embed`

Instead of making HTTP calls to the AEM servlet, the commerce system places an `<aem-embed>` Web Component tag on the product page. This component fetches the DA document, runs the EDS decoration pipeline, and renders the content inside an isolated Shadow DOM — preventing any CSS or JS conflicts between the pod content and the commerce page.

DA Mirrors the AEM Hierarchy

Just as pods exist at global, regional, and country levels in AEM, they exist at the same levels in DA. DA MSM inheritance handles fallback — if a pod does not exist at country level, DA automatically serves the regional version; if no regional version exists, it falls back to global.

3. The Fundamental Change — Tag to Path

AEM Today

The AEM path of a Content Snippet page has no relationship to the product it serves. The link is purely through `cq:tags` :

```
1 Page path: /content/lifetech/ipac/en-in/syndicated-
2 content/snippets/cell-culture/promotions/explore-gibco.html
3 cq:tags:    pfp:sku/CHROMELEON7/pod_3
```

The page can sit anywhere in the folder tree. The tag is what identifies it.

DA After Migration

The DA document path IS the identifier. No tags, no servlet lookup:

```
1 DA path:    ipac/en-in/snippets/pfp/sku/CHROMELEON7/pod-3
2
```

Path Derivation Rule

The DA target path is derived from two inputs: the **AEM locale** (extracted from the page's position in the JCR tree) and the **cq:tag** value.

```
1 AEM locale from page path + cq:tag value
2   ↓                       ↓
3 {da-site-path}           + snippets/{namespace}/{type}/{SKU}/pod-{N}
4   ↓
5 DA target path
6
```

Input	Example
AEM page path	/content/lifetech/ipac/en-in/syndicated-content/...
cq:tag	pfp:sku/CHROMELEON7/pod_3
Derived DA site path	ipac/en-in
Derived DA fragment path	ipac/en-in/snippets/pfp/sku/CHROMELEON7/pod-3

The namespace (`pfp` or `pdp`) is preserved in the DA path so that Product Family Page pods and Product Detail Page pods remain distinguishable in DA.

4. DA Path Structure

Full DA Path Examples

AEM Locale	cq:tag	DA Target Path
global/en	pfp:sku/CHROMELEON7/pod_1	global/en/snippets/pfp/sku/CHROMELEON7/pod-1
north-america/en	pfp:sku/CHROMELEON7/pod_1	north-america/en/snippets/pfp/sku/CHROMELEON7/pod-1
ipac/en-in	pfp:sku/CHROMELEON7/pod_3	ipac/en-in/snippets/pfp/sku/CHROMELEON7/pod-3
europa/de-de	pdp:sku/ABC123/pod_1	europa/de-de/snippets/pdp/sku/ABC123/pod-1
japan/ja-jp	pfp:fam/CELL-CULTURE/pod_2	japan/ja-jp/snippets/pfp/fam/CELL-CULTURE/pod-2

5. Migration Approach

Pods Migrate with Their Locale Wave

Pods are migrated in the same wave as the locale they belong to — when a country or region is being migrated, its syndication pods are migrated as part of that same wave.

```
1 Wave 1 – global/en migration
2   → global/en pages migrated
3   → global/en syndication pods migrated
4
5 Wave 2 – Regional EN hubs migration
6   → north-america/en pages migrated
7   → north-america/en syndication pods migrated
8   → (same for europa/en, ipac/en, etc.)
9
10 Wave 3 – Country EN sites migration
11   → north-america/en-us pages migrated
12   → north-america/en-us syndication pods migrated
13   → (same for all country EN sites)
14
15 Wave 4 – Translated locales migration
16   → (same for all translated locales)
17
```

How Pods Are Found

For each locale being migrated, the migration script queries AEM for all Content Snippet pages under that locale's path, using the Content Snippet template as the filter.

Tag Handling Rules

Once pod pages are found, their `cq:tags` determine the DA target path. The following rules govern how tags are handled:

Tag situation	Handling
Page has tag	Derive DA path from that tag, migrate to that path
Page has multiple tags	Migrate to each tag's corresponding DA path — same content published at multiple paths
Page has no tags	Skip — DA path cannot be derived, pod is excluded from migration

6. Migration Steps

Step 1 — Query AEM for All Pod Pages

For the locale being migrated, query AEM to find all Content Snippet pages under that locale's `syndicated-content/snippets/` tree using the `lifetech/templates/content_snippet` template. This produces a complete list of pod pages for that locale.

Step 2 — Extract Tags and Derive DA Paths

For each pod page found, read its `cq:tags` property. Apply the path derivation rule (locale + tag → DA path) to produce the target DA path for each pod. Pages with no tags are excluded. Pages with multiple tags produce one DA path per tag.

This produces the pod migration manifest for that locale — a complete list of source AEM pages mapped to their DA target paths.

Step 3 — Fetch Pod HTML

For each pod in the manifest, fetch the rendered HTML from the AEM publish instance using the `.pfpsnippet.html` selector. This renders the Content Snippet page as a clean HTML

fragment — the same output the commerce system receives today. No page shell, no navigation, no CSS — pure content HTML ready for transformation.

Step 4 — Transform to DA Format

The scraped HTML is transformed into DA block table format and wrapped in a complete HTML document. The content of the pod — headings, paragraphs, images, video, links — is preserved as authored. The transformation converts the AEM component output into the block structure that DA stores and EDS renders.

Step 5 — Push to DA

The transformed document is pushed to DA via the DA Admin API at the derived target path. Once pushed and published, the pod is immediately available at its DA URL and can be fetched by the `aem-embed.js` Web Component.

7. Pilot and Cutover

Pilot — One SKU End to End First

Before migrating all pods, the pipeline is validated end to end with a single SKU in coordination with the commerce team. All pods for that one SKU are migrated to DA . The commerce team then switches that SKU's pod calls from the AEM servlet URL to `<aem-embed>` tags pointing to the DA paths.

If the pilot SKU renders correctly in the commerce page — content, styling, locale resolution — the full migration runs for all remaining pods.

Full Cutover

After pilot sign-off, all remaining pods are migrated locale by locale within each migration wave. Once all pods for a locale are in DA, the commerce team updates their system for that locale to use `<aem-embed>` tags. The AEM servlet remains available as a fallback until the commerce team confirms the switch is complete for all locales.

8. Before and After

AEM — Current State

```
1 Commerce system calls:
2   /{locale}/product-family-syndicated-
   content.pfpsnippet.html/sku/CHROMELEON7/3
3
4 AEM servlet receives the call
5   → Parses SKU and pod number from URL
```

```

6   → Builds tag: pfp:sku/CHROMELEON7/pod_3
7   → Searches JCR by tag within locale scope
8   → Finds tagged Content Snippet page (wherever it sits in folder
   tree)
9   → Renders page via content-snippet.jsp
10  → Returns raw HTML fragment to commerce system
11
12 Content Snippet page in AEM:
13   Path:    /content/lifetech/ipac/en-in/syndicated-
   content/snippets/cell-culture/promotions/explore-gibco.html
14   Tag:     pfp:sku/CHROMELEON7/pod_3
15   Content: text, images, links authored via AEM drag-and-drop
   components
16

```

DA — After Migration

```

1 Commerce system places:
2   <aem-embed url="{locale}/snippets/pfp/sku/CHROMELEON7/pod-3"></aem-
   embed>
3
4 aem-embed.js Web Component runs in the browser
5   → Reads the url attribute
6   → Fetches DA document as plain HTML fragment
7   → DA MSM resolves locale – serves locale-specific pod if it exists,
   falls back up hierarchy if not
8   → Renders content in isolated Shadow DOM – no CSS/JS conflicts with
   commerce page
9
10 Pod document in DA:
11   Path:    ipac/en-in/snippets/pfp/sku/CHROMELEON7/pod-3
12   Content: same content, authored as DA block tables
13   No tags. No servlet. No JCR search.
14

```

What Changes — Summary

Dimension	AEM	DA
Content location	Descriptive folder path, identified by tag	Path IS the identifier — no tags
Commerce integration	HTTP call to AEM servlet	<code><aem-embed></code> Web Component tag
Locale resolution	Server-side Java (<code>MsmMappingConfigurati on</code>)	DA MSM — platform- level fallback
Content lookup	JCR tag search (<code>TagManager.find()</code>)	Direct URL resolution
CSS isolation	None — commerce CSS must not conflict	Full Shadow DOM isolation

Migration Strategy -- Reference Component

Overview

This document defines the migration strategy for AEM Reference Components (`foundation/components/reference`) from AEM 6.4 to DA/EDS.

Reference Components are AEM's content reuse mechanism — a page embeds a specific component that physically lives on a dedicated reference-components source page, rather than duplicating content across pages. In DA/EDS, content reuse is achieved through DA fragments. The migration strategy converts each referenced component into a DA fragment and replaces the reference on every consuming page with a fragment include block.

1. What Are Reference Components

The Pattern in AEM

TFS uses dedicated **reference-components pages** in AEM to store reusable content — primarily raw HTML embeds, scripts, and structured content blocks that need to appear consistently across multiple pages. Instead of duplicating this content on every page that needs it, pages embed it using the `foundation/components/reference` component, which points to the specific component node on the source page by its JCR path.

```
1 reference-components/
2   └─ gds/           ← source page holding reusable components
3   └─ (others...)    ← other source pages per use case
4
5 Consuming page A
6   └─ MainParsys
7       └─ reference_ad2b [foundation/components/reference]
8           path → ../reference-components/gds/.../ltrawhtml_519f
9       └─ reference_fa59 [foundation/components/reference]
10          path → ../reference-components/gds/.../ltrawhtml_def9
11
12 Consuming page B
13   └─ MainParsys
14       └─ reference_ad2b [foundation/components/reference]
15          path → ../reference-components/gds/.../ltrawhtml_519f
16          (same component, same source – single source of truth)
17
```

One component in the source page is shared across as many consuming pages as needed. A change to the source updates everywhere it is referenced.

How It Renders on Live Pages

On the live site, the AEM render engine outputs a wrapper div with the CSS class `reference` `parbase` `section` around the referenced component's rendered output. This class is the

identifiable signal in scraped HTML that a reference component is present on the page.

Live example:

```
1 https://www.thermofisher.com/us/en/home/about-  
2 us/events/industrial/amira-software-capabilities-bones-tissue.html
```

In the browser DevTools Elements panel on this page:

```
1 <div class="reference parbase section">  
2   <div class="cq-dd-paragraph">  
3     <div class="ltrawhtml_def9 LTRawHTML">  
4       <!-- referenced component content rendered here -->  
5     </div>  
6   </div>  
7 </div>  
8
```

The inner class (`ltrawhtml_def9`) reflects the JCR node name of the referenced component on the source page.

2. What Changes in DA

In AEM, the reference component is a pointer — it resolves at render time to a component on another page. Content reuse in DA is done through **DA fragments** — standalone documents that any page can include via a fragment block.

The migration replaces every AEM reference component with a DA fragment include block. The referenced content becomes a DA fragment document. Single source of truth is fully preserved — the fragment is authored once in DA and included on as many pages as needed, exactly as today.

3. Detection — How the Migration Script Identifies Reference Components

Step 1 — HTML detection

The migration script scans the scraped page HTML for the CSS class `reference parbase section` . If found, the page contains one or more reference components.

Step 2 — JCR lookup

The rendered HTML confirms a reference component is present but does not contain the source path it points to. The script queries the JCR for that page to retrieve the `path` property of each `foundation/components/reference` node. This `path` is the JCR address of the actual component being referenced — the source of the DA fragment.

```

1 HTML scan → "reference parbase section" found
2           ↓
3 JCR query → get path property of each reference node
4             e.g. /content/lifetech/north-america/en-us/
5                 reference-
6 components/gds/jcr:content/MainParsys/ltrawhtml_519f

```

4. DA Fragment Path Derivation

The DA fragment path is derived from the AEM `path` property by a deterministic rule — no pre-built mapping table is required.

Derivation Rule

```

1 AEM path:
2 /content/lifetech/{region}/{locale}/reference-
3 components/{name}/jcr:content/MainParsys/{component-node}
4
5 Strip /content/lifetech/ prefix
6 Strip /jcr:content/MainParsys/ middle segment
7 Normalise underscores to hyphens in component node name
8 Insert /fragments/ after locale
9
10 DA fragment path:
11 {region}/{locale}/fragments/reference-components/{name}/{component-
12 node}

```

Examples

AEM Reference Path	DA Fragment Path
/content/lifetech/north-america/en-us/reference-components/gds/jcr:content/MainParsys/ltrawhtml_519f	north-america/en-us/fragments/reference-components/gds/ltrawhtml-519f
/content/lifetech/north-america/en-us/reference-components/gds/jcr:content/MainParsys/ltrawhtml_def9	north-america/en-us/fragments/reference-components/gds/ltrawhtml-def9
/content/lifetech/global/en/reference-components/gds/jcr:content/MainParsys/ltrawhtml_519f	global/en/fragments/reference-components/gds/ltrawhtml-519f

The locale of the source reference-components page determines the locale of the DA fragment — ensuring the correct locale fragment is used for each site.

5. Migration Steps

Step 1 — Scrape Page HTML

Standard page scraping. The full page HTML is fetched.

Step 2 — Detect Reference Components

Script scans the scraped HTML for `class="reference parbase section"`. If found, the page contains one or more reference components. The count of matches tells the script how many reference nodes to expect in JCR.

Step 3 — Query JCR for Reference Paths

For each `reference parbase section` div detected, the script queries the JCR of the page being migrated to retrieve the `path` property of the corresponding `foundation/components/reference` node. This gives the full AEM source path.

Step 4 — Derive DA Fragment Path

Apply the derivation rule to each AEM source path to compute the corresponding DA fragment path.

Step 5 — Create or Reuse DA Fragment

For each derived DA fragment path:

```
1 | Does this DA fragment already exist?
2 |   YES → skip creation, proceed to Step 6
3 |   NO  → fetch component content from JCR at the source path
4 |         → transform to DA block format
5 |         → push to DA as a new fragment document at the derived path
6 |         → mark as created (cached for remainder of migration run)
7 |
```

This ensures the same fragment is never created twice regardless of how many pages reference it.

Step 6 — Replace Reference Div with Fragment Block

In the scraped HTML, replace each `<div class="reference parbase section">` with a DA fragment include block pointing to the corresponding DA fragment path.

Step 7 — Continue Page Assembly and Push to DA

The page with reference components replaced by fragment blocks is assembled as normal and pushed to DA at the mapped page path.

7. Before and After

AEM — Current State

```
1 Consuming page JCR:
2   └─ MainParsys
3       └─ reference_ad2b [sling:resourceType:
4         foundation/components/reference]
5           path: /content/lifetech/north-america/en-us/
6             reference-
7 components/gds/jcr:content/MainParsys/ltrawhtml_519f
8   └─ reference_fa59 [sling:resourceType:
9     foundation/components/reference]
10      path: /content/lifetech/north-america/en-us/
11        reference-
12 components/gds/jcr:content/MainParsys/ltrawhtml_def9
13
14 Rendered HTML:
15 <div class="reference parbase section">
16   <div class="ltrawhtml_519f LTRawHTML"> ... </div>
17 </div>
18 <div class="reference parbase section">
19   <div class="ltrawhtml_def9 LTRawHTML"> ... </div>
20 </div>
21
22 Source of truth: reference-components/gds page in AEM Sites
23 Reuse mechanism: foundation/components/reference pointer resolved at
24 render time
```

DA — After Migration

```
1 DA fragment documents created:
2   north-america/en-us/fragments/reference-components/gds/ltrawhtml-
3   519f
4   north-america/en-us/fragments/reference-components/gds/ltrawhtml-
5   def9
6
7 Consuming page DA document:
8   [fragment block]
9   path: north-america/en-us/fragments/reference-
10  components/gds/ltrawhtml-519f
11
12   [fragment block]
13   path: north-america/en-us/fragments/reference-
14  components/gds/ltrawhtml-def9
15
16 Source of truth: DA fragment documents
17 Reuse mechanism: Fragment include block – same fragment referenced by
18 any number of pages
```

What Changes — Summary

Dimension	AEM	DA
-----------	-----	----

Content storage	Component node on reference-components source page	DA fragment document
Reuse mechanism	<code>foundation/components/reference</code> pointer by JCR path	Fragment include block by DA path
Detection in HTML	<code>class="reference parbase section"</code>	Fragment block in DA document
Source path lookup	JCR <code>path</code> property on reference node	DA fragment path derived by rule
Single source of truth	Yes — one AEM component, many pages	Yes — one DA fragment, many pages
Author update	Edit component on source reference-components page	Edit DA fragment document

Migration Strategy -- Redirects

Overview

This document defines the migration strategy for TFS redirects from AEM 6.4 to EDS. It covers how redirects work in EDS, what types of redirects exist in AEM today, and clearly defines who owns what in the migration process.

EDS Official Reference: [🔗 Redirects](#)

1. How Redirects Work in EDS

In EDS, redirects are managed through a single spreadsheet file — `redirects.xlsx` — stored at the **root of the project**. There are no server-side redirect configurations, no OSGi services, and no Dispatcher rules. All simple redirects are managed in one place by content authors.

Spreadsheet Format

The spreadsheet must contain exactly two columns:

Source	Destination
<code>/old-page-path</code>	<code>/new-page-path</code>
<code>/another-old-path</code>	<code>https://external-site.com/page</code>

- **Source** — relative path from the domain root (e.g. `/us/en/old-page`)
- **Destination** — relative path for internal redirects or a fully qualified URL for external redirects
- When multiple sheets exist in the workbook, EDS only processes the sheet named `helix-default`

Important Behaviour

- EDS redirects are intended to handle **permanent redirects (HTTP 301)** — the spreadsheet mechanism is designed for permanent URL changes, not temporary redirects
- Redirects **take precedence over existing content** — if a redirect is configured for a path, it is served even if a published page exists at that path
- Redirects are managed and published — changes can be previewed on the `.page` environment before going live

- No code or configuration deployment is required — authors update the spreadsheet and publish via Sidekick
- The spreadsheet is the single source of truth for all simple redirects in EDS

Wildcard and Pattern-Based Redirects

Pattern-based wildcard redirects are **not supported** in the `redirects.xlsx` spreadsheet. These must be handled at the CDN level (Akamai for TFS). CDN wildcard syntax uses `/old-path/*` redirecting to `/new-path/$1`, where `*` captures the remaining URL segment and `$1` represents it in the destination.

2. Types of Redirects in AEM Today

TFS currently manages redirects through three mechanisms in AEM:

2.1 Page-Level Redirects (Page Properties)

Authors can configure a redirect directly on a page via the **Advanced tab** of page properties. When set, accessing that page's URL redirects the visitor to the configured destination.

2.2 Vanity Paths

Authors can assign short or alternative URLs to any page via page properties. A visitor accessing the vanity URL is served that page's content.

2.3 Dispatcher / CDN Pattern Redirects

Pattern-based and wildcard redirects are configured at the Dispatcher and CDN (Akamai) level — outside the AEM content tree. These cover bulk URL patterns, domain-level redirects, and path-based rewrite rules that cannot be expressed as simple 1:1 redirects.

3. Migration Approach

Step 1 — TFS Provides Complete Redirect Inventory

TFS must compile and provide the complete list of all active redirects across all three mechanisms above. This is a **TFS ownership**. The inventory must cover:

- All page-level redirects (cq:redirectTarget)
- All vanity paths (sling:vanityPath)
- All Dispatcher and CDN pattern-based redirects

Step 2 — Categorise Redirects

Once the inventory is provided, redirects are split into two categories:

Category	Criteria	Target in EDS
Simple 1:1 redirects	Single source path → single destination path	<code>redirects.xlsx</code> spreadsheet
Pattern / wildcard redirects	URL patterns, wildcards, bulk path rewrites	Akamai CDN configuration

Step 3 — Simple Redirects into EDS Spreadsheet

Simple 1:1 redirects are formatted into the Source / Destination columns of `redirects.xlsx` and placed at the project root in DA. Adobe supports the setup and publishing of the spreadsheet.

Step 4 — Pattern Redirects to Akamai

Wildcard and pattern-based redirects are passed to the TFS Akamai infrastructure team for CDN-level configuration. These are outside the EDS spreadsheet mechanism and outside Adobe's migration scope.

4. Ownership

Activity	Owner
Compile complete redirect inventory (all types)	TFS
Confirm which redirects are active and in scope	TFS
Categorise redirects (simple vs pattern)	TFS + Adobe
Set up and publish <code>redirects.xlsx</code> in EDS	Adobe
Configure wildcard / pattern redirects in Akamai	TFS Infrastructure / Akamai team

Validate redirects post go-live	TFS
---------------------------------	------------

5. Open Items

Item	Owner	Status
Complete redirect inventory provided	TFS	Open
Total volume of redirects	TFS	Open
Akamai team engaged for CDN redirects	TFS Infrastructure	Open

Migration Strategy -- Metadata

Overview

This document defines the migration strategy for page metadata from AEM 6.4 to EDS as part of the broader TFS migration. It covers how metadata works in EDS, what metadata exists on TFS pages today, how it is extracted and migrated, and who owns what.

EDS Official References:

- [🔗 Metadata Block](#)
 - [🔗 Page Metadata](#)
 - [🔗 Bulk Metadata](#)
-

1. How Metadata Works in EDS

EDS supports two mechanisms for managing metadata — page-level and bulk. Both use the same property names and serve the same purpose, but differ in scope and where they are authored.

Page-Level Metadata

Page-level metadata is defined inside each DA document as a metadata table placed at the bottom of the page. The table has two columns — a property name and its value — and the first row uses the header "Metadata" to identify the block. EDS reads this table and generates the corresponding HTML `<head>` tags when the page is served.

This is per-page — it applies only to the document it is defined in.

Bulk Metadata

Bulk metadata is managed through a single `metadata.xlsx` spreadsheet at the project root. The first column is the URL (which supports wildcard patterns to match entire sections of the site), and subsequent columns are property names. EDS evaluates rows top-to-bottom and the first matching URL pattern wins. Page-level metadata always takes priority over bulk metadata when the same property is defined in both places.

This is used for site-wide or section-wide defaults — properties that apply consistently across a path pattern and do not need to be defined on every individual page.

2. What Metadata Exists on TFS Pages Today

TFS pages currently serve metadata as HTML `<head>` tags rendered by AEM. Based on observation of live TFS pages, the following types of metadata tags are present in the page HTML today:

Category	Sample Tags Observed
Page identity	Page title, meta description
SEO	Robots directives, canonical URL
Open Graph	og:title, og:description, og:image, og:type, og:locale, og:url
Site verification	Third-party site verification tags
Other	DC.title, additional custom meta tags

Sample reference:

<https://www.thermofisher.com/in/en/home.html>

This is an observation of what currently exists — not a confirmed migration scope. **TFS must confirm which metadata properties should be carried forward into EDS.** The migration script will extract the confirmed properties from the scraped page HTML and write them into the metadata table in the DA document. Properties not confirmed by TFS will not be migrated.

EDS Supported Property Names

EDS recognizes standard metadata property names directly. The property names used in the metadata table map to the corresponding HTML `<head>` tags when the page is served. EDS supports standard properties (title, description, robots, canonical-url, image), Open Graph properties using the `og:` prefix, and custom properties defined by the project.

3. Migration Approach

Source — Page HTML Scraping

Metadata is extracted directly from the `<head>` section of the scraped page HTML. The rendered HTML served by AEM already contains all metadata tags exactly as they appear to users and search engines today. There is no need to look up JCR properties separately — the scraped HTML is the source of truth.

This keeps the metadata migration fully consistent with the rest of the page migration approach and ensures what is migrated reflects what is actually being served on the live site.

How It Works

When the migration script scrapes a page, it reads the `<head>` section alongside the page body. The confirmed metadata properties are extracted, mapped to their EDS property names, and written into a metadata table appended at the bottom of the DA document. The page and its metadata are pushed to DA together in a single operation — metadata migration is not a separate workstream.

Bulk Metadata — Implementation Efficiency

Where the same metadata property applies consistently across many pages — for example, the same robots setting across an entire section, or the same og:type across all pages of a given type — bulk metadata is used rather than writing that property into every individual DA document. This is an implementation decision that keeps the migration clean and makes future site-wide changes easier to manage.

Page-level metadata is always migrated per page, and bulk metadata handles consistent values at scale.

4. Ownership

Activity	Owner
Metadata extracted from scraped HTML and written to DA document	Adobe
Define which metadata properties are in scope for migration	TFS
Identify bulk metadata patterns and configure <code>metadata.xlsx</code>	Adobe
Validate metadata on migrated pages post go-live	TFS

5. Open Items

Item	Owner	Status
------	-------	--------

Confirm which metadata properties should be included in migration	TFS	Open
---	-----	-------------

Assets Migration

Overview

This document defines the migration strategy for TFS DAM assets from AEM 6.4 to AEM as a Cloud Service as part of the broader AEM to EDS + DA migration. The approach is a 1:1 as-is migration — assets are transferred as they exist today without any structural reorganisation or cleanup as part of the migration scope.

1. Migration Approach

Content Transfer Tool (CTT)

DAM assets are migrated to AEM as a Cloud Service using the **Content Transfer Tool (CTT)**. CTT is the Adobe-recommended mechanism for migrating content from AEM 6.4 to AEM Cloud and handles the transfer of assets, folder structures, renditions in a single run.

The migration is performed as a **1:1 as-is transfer** — what exists in AEM 6.4 DAM today is replicated to AEM Cloud DAM exactly as it is. No restructuring, renaming, deduplication, or cleanup is performed as part of the migration run.

2. What Is Included in Scope

DAM Assets

All active DAM assets under `/content/dam/` that are referenced by live site pages — images, documents, videos, and associated renditions and metadata.

Content Fragments

Content Fragments stored in DAM are explicitly included in the migration scope and are called out separately due to their functional dependency on site components.

Content Fragments used by TFS include:

Content Fragment Location	Used By	Purpose
---------------------------	---------	---------

/content/dam/form commons/cf/	Form components	Populates field options such as countries, divisions, and other structured data
----------------------------------	-----------------	---

These Content Fragments are not just assets — they are functional dependencies that components actively query at runtime. They must be present and verified in AEM Cloud before any component or form testing begins.

3. What Is Not in Scope

Activity	Owner	Notes
DAM folder structure redesign	TFS	Not part of migration scope
Asset cleanup and deduplication	TFS	Not part of migration scope
Taxonomy and governance model definition	TFS	Not part of migration scope
Metadata schema changes	TFS	Not part of migration scope

These activities are part of a broader DAM governance initiative and are owned by TFS. They can be completed either before or after migration depending on TFS preference — see Section 4.

4. DAM Governance — TFS Responsibility

DAM structure and governance decisions sit entirely with TFS. The migration does not prescribe or enforce any particular folder structure or taxonomy. TFS has two options:

Pre-migration cleanup TFS restructures, deduplicates, and aligns the DAM to the desired target state before CTT runs. Migration then carries over the already-clean structure. This is the preferred approach if TFS has a defined target DAM structure ready.

Post-migration cleanup Assets are migrated as-is. TFS performs controlled restructuring and cleanup in AEM Cloud after migration completes. This approach accepts some technical debt in the short term but avoids blocking the migration timeline.

Adobe will not make DAM governance decisions on TFS's behalf — content ownership sits with TFS.

5. Asset Volume

The exact volume of assets to be migrated is currently unknown. This needs to be confirmed by TFS.

Information Required	Owner	Status
Total number of assets under <code>/content/dam/</code>	TFS	Open — to be confirmed
Total size of DAM (GB)	TFS	Open — to be confirmed

Volume directly impacts CTT run time and migration window planning. This must be provided by TFS as a prerequisite to finalizing the migration schedule.

Note: Adobe's current access is limited to the TFS QA environment. Asset volume figures from QA are not representative of production and cannot be used for migration planning. Accurate volume numbers must come from the TFS production DAM to be provided by TFS.

Content Management & Localization

TFS manages content across a four-level hierarchy — Global → Region → Country → Language — spanning 27 English sites and 13 language alternate sites globally. In DA, multi-site inheritance, translation, and content reuse through fragments all operate on the same underlying model. This section documents how each of these capabilities is addressed for TFS.

TFS Content Hierarchy

Level	Visibility
Global English	Internal — master content source
Regional English	Internal — e.g. North America, Europe, Latin America
Country English	Public — e.g. US English, Brazil English, UK English
Local Language	Public — e.g. Brazil Portuguese, French, German

What This Section Covers

Child Page	Covers
Multi-Site Manager (MSM)	DA MSM model, TFS hierarchy mapping, functional use cases — inheritance, country fallback, rollout, relocation, visualization
Translation	DA translation approach, TFS localization workflow, WordBe connector
Experience Fragments	How XFs work in DA, MSM-based fragment resolution

Multi-Site Manager (MSM)

DA Official Reference: [Multi-Site Manager](#)

Overview

This document defines how Multi-Site Manager (MSM) is addressed in DA for TFS. It covers how MSM works in DA, how TFS's content hierarchy maps to the DA MSM model, and how each functional requirement is addressed — whether out of the box, through a process change, or through custom development.

1. DA MSM — Current Capability and TFS Collaboration

DA MSM is an actively evolving capability within the DA platform. The Adobe engineering team building DA is continuously expanding MSM to support increasingly complex multi-site requirements.

In its current state, DA MSM supports **one level of inheritance** — a base site rolling out directly to a satellite site. TFS's content hierarchy requires **three levels of inheritance** — Global → Region → Country → Language. To support this, the Adobe DA engineering team will work in direct collaboration with TFS to extend the MSM capability to support TFS's three-level inheritance model. Given TFS's scale — 27 English sites and 13 language alternates across a global hierarchy.

This collaborative engagement means TFS gets direct involvement from the engineering team building the product, with the solution shaped specifically around TFS's real-world hierarchy and authoring needs.

2. How MSM Works in DA

Understanding DA MSM requires a shift in mental model from AEM Sites. The two systems handle inheritance in fundamentally different ways.

AEM Sites — Content Duplication Model

In AEM Sites, when a page is rolled out from a parent to a child site, the content is physically duplicated. Rolling out Global English to North America English creates an identical copy of every page in the North America folder. From that point, authors manage two separate sets of content — the original in Global English and the copies in North America English. Any

inheritance relationship is maintained through MSM sync, but the content itself exists in both places.

DA — File Existence Model

In DA, inheritance is determined entirely by whether a file exists in the satellite site's folder. There is no duplication.

- If a page **does not exist** in the satellite folder — the satellite inherits that page directly from the base site. The page is served to visitors from the base, transparently, at delivery time. No copy sits in the satellite folder.
- If a page **does exist** in the satellite folder — it is treated as a local override. The satellite's version is served instead of the base version.

This means when a base site is rolled out to a satellite, the satellite folder starts completely empty. Authors only see content in the satellite folder when they have explicitly broken inheritance for a specific page. Everything else flows silently from the base.

What This Means for TFS

Today, TFS's MSM setup in AEM creates thousands of page copies across every region and country level. In DA, those copies do not exist by default. Only the pages that TFS has actively localised — where an author has made a region- or country-specific change — will appear in the corresponding DA folder. This is a cleaner, lower-overhead model for content management at TFS's scale.

Two Author Roles in DA MSM

Role	Responsibilities
Base site author	Manages shared content, rolls out changes to satellite sites, cancels or resumes inheritance for specific pages, syncs updates to satellites
Satellite site author	Works on locally overridden pages, syncs updates from base when needed, resumes inheritance to revert a page back to the base

3. TFS Content Hierarchy

TFS organizes content across four levels. The hierarchy flows from a single global content source down through regions, countries, and local language variants.

1	Global English (non-public – master content source)
2	└─ Region English (non-public – e.g. North America, Europe, Latin America)
3	└─ Country English (public – e.g. US English, Brazil English, UK English)
4	└─ Local Language (public – e.g. Brazil Portuguese, French, German)
5	

- **27 English sites** are publicly accessible (country-level English)
- **13 language alternate sites** are publicly accessible (translated local language)
- Global and Regional English are internal — not served to public visitors

This four-level structure maps directly to the DA MSM model as a multi-level base-satellite hierarchy, with Global English as the root base site and each subsequent level acting as both a satellite to its parent and a base to its children.

4. Functional Use Cases — How Each Is Addressed in DA

The following covers each of TFS's functional MSM requirements, how they are met in DA, and what — if anything — changes from today.

Use Case 1 — Multi-Level Content Cascade (Global → Region → Country → Language)

What TFS does today: Content authored at Global English level is cascaded through custom MSM rollout services to Regional English, then Country English, then local language variants. This multi-level cascade is not natively available in AEM Sites — TFS built and maintains a custom service to achieve it.

How it works in DA: Multi-level inheritance is being built into the DA product natively. Once available, the entire cascade — Global through to local language — will work out of the box without any custom development. The product team is actively building this capability, with TFS's use case directly informing the scope.

Status: OOTB — pending DA product delivery of multi-level inheritance

Use Case 2 — Authoring at All Levels and Inheritance Management

What TFS does today: Authors at regional and country levels can create net new pages at their level without starting from global. They can also break MSM inheritance either at the full page level or at an individual component level, allowing localized content while the rest of the page continues to flow from the parent.

How it works in DA:

Creating pages at any level — OOTB Authors can create net new pages at any level of the hierarchy — regional, country, or language — without the page needing to exist at the global level first. This is fully supported.

Breaking page-level inheritance — OOTB Two mechanisms are available:

- A base site author can cancel inheritance for a specific satellite page — the page is then copied into the satellite folder as a local override
- A satellite author can create a page at the same path in their folder — DA treats the presence of that file as an implicit inheritance break

Either way, once the page exists in the satellite folder, it is locally managed from that point onwards. The reverse is also available — authors can use **Resume Inheritance** to delete the local override and revert the page back to inheriting from the base. Inheritance breaks are not permanent.

Breaking block-level inheritance — Process Change

DA does not support breaking inheritance at the block or component level while keeping the rest of the page inherited. This is a process change from AEM Sites.

Process change — how it works in DA: Authors break the inheritance of the full page (as described above). Once the page is a local override, they use the **Sync from Base** action with **Merge** mode — DA shows a content diff of what has changed in the base page versus the local version. Authors review the diff block by block and choose which base changes to accept and which to keep as localised. For the block they want to keep localised (e.g. a regional hero banner), they reject the incoming change. For everything else, they accept it. The result is the same outcome — one block is localised, the rest reflects the parent — but achieved through a sync-and-review step rather than a component-level toggle.

Status: OOTB for page-level and net new pages. Process change for block-level inheritance.

Use Case 3 — Country Fallback / Virtual Sites (e.g. Belgium → UK)

What TFS does today: Countries without their own dedicated site nodes — such as Belgium — are mapped to a primary country through a custom configuration service. Belgium is configured to serve the UK site. No Belgium site is maintained in the content tree; the custom service handles the routing transparently.

How it works in DA: The custom configuration service approach is replaced by a direct rollout. Belgium is rolled out from the Europe region as a satellite site, exactly as any other country would be. Because DA does not duplicate content, the Belgium folder starts empty — no pages are created, no authoring overhead is introduced. Belgium content flows directly from Europe region at delivery time, achieving the same end result as today without a custom routing service.

Process change: Country-to-country inheritance (Belgium inheriting from UK) should be replaced by country inheriting from its region (Belgium inheriting from Europe). This is the recommended content organization pattern in DA — cleaner hierarchy, no custom configuration to maintain.

Status: Process change — rollout from region instead of custom configuration service

Use Case 4 — Custom Recursive Rollout

What TFS does today: TFS maintains a custom MSM rollout service that overrides the AEM OOTB live copy resolution. When a rollout is triggered, this service recursively walks the entire content tree, discovers all direct and indirect live copies, and builds a dependency-ordered rollout queue. Country language variants wait for their country English parent to complete before rolling out.

How it works in DA: This entire customization exists because AEM Sites does not support multi-level inheritance natively. Once DA delivers multi-level inheritance in the product, recursive rollout across all levels is handled automatically. There is no custom service to maintain.

Status: OOTB — with DA multi-level inheritance delivery. Custom service not required.

Use Case 5 — Automatic Path and Link Rewriting During Rollout

What TFS does today: When content is rolled out from Global English to a regional or country node, all internal links and asset path references within the page are automatically rewritten to reflect the target region's path structure. TFS maintains this as a custom capability because AEM Sites does not handle it natively.

How it works in DA: As with recursive rollout, this capability is a natural part of how multi-level inheritance functions in DA. When a page is inherited at a given level, path references resolve correctly for that level's context without manual rewriting or a custom service.

Status: OOTB — with DA multi-level inheritance delivery. Custom service not required.

Use Case 6 — "Other" Nodes (Catch-All for Unmapped Countries)

What TFS does today: Regions maintain "Other" nodes (e.g. Latin America Other, Europe Other) that serve as catch-all destinations for countries not mapped to a specific primary country. These are managed through the same custom configuration service used for country fallback.

How it works in DA: Same approach as Use Case 3. These catch-all groupings are handled by rolling out the relevant countries from their parent region directly. Since DA does not duplicate content, creating these satellite sites has no authoring overhead. The "Other" grouping concept is replaced by regional inheritance.

Status: Process change — same approach as country fallback

Use Case 7 — Activation Cascade (Global → Region → Country)

What TFS does today: When a page is activated at the global level, it triggers activation for all corresponding regional and country nodes sequentially, one by one.

How it works in DA: Publishing in DA MSM cascades to all satellite levels as part of the multi-level inheritance model. A base site author publishing a page pushes it to preview or live environments across all inheriting satellite sites. This is handled natively.

Status: OOTB

Use Case 8 — Custom Relocation (Move with Redirect and Child Pages)

What TFS does today: TFS has a custom relocation function that goes beyond a standard page move. When a page is relocated, all its live copies across regions and countries move with it. The source page is kept in place (not deleted) and configured to redirect to the new location — ensuring bookmarked URLs and external links continue to work. Authors can also include child pages in the relocation.

How it works in DA: This specific relocation behavior — keeping source as a live redirect while copying to the new location across all levels — is not available out of the box in DA. It requires a **custom plugin or app** built using the DA SDK.

Without the customization: When an author relocates a page, they manually add an entry to the redirects spreadsheet (redirects.xlsx) to create the redirect from the old path to the new location. This achieves the same end result for visitors but requires an explicit authoring step rather than an automated action.

With the customization: The relocation action in DA is extended via a custom plugin that automates the redirect creation and handles the cascade across all satellite sites, matching the current behavior.

The custom plugin is an enhancement — the path to EDS is not blocked without it.

Status: Customization required — DA plugin/app. Manual redirect entry available without customization.

Use Case 9 — MSM Visualization (Multi-Level Relationship View)

What TFS does today: TFS maintains a custom interface that visualizes the full MSM relationship tree — direct and indirect live copy relationships, inheritance status, sync flags, and which pages have broken inheritance. AEM's default MSM view only shows first-level relationships.

How it works in DA: A basic MSM relationship view is available out of the box in DA, showing the base-satellite relationships and whether pages are inherited or locally overridden. For TFS's full multi-level visualization — matching the depth and detail of the current custom interface — a **custom plugin or app** can be built using the DA SDK.

Without the customization: The OOTB view provides enough visibility for standard authoring and inheritance management. Authors can identify which pages are overrides and which are inherited.

With the customization: The view is extended to match TFS's current reporting depth, including multi-level relationship tracking and inheritance status across all levels.

Status: Basic view OOTB. Enhanced multi-level view requires custom plugin/app.

5. Summary

Use Case	Status	Notes
Multi-level content cascade (Global → Region → Country → Language)	OOTB — pending product	DA product roadmap. Key dependency for entire MSM solution.
Net new pages at any level	OOTB	Fully supported
Page-level inheritance break and Resume Inheritance	OOTB	Two mechanisms available. Break is reversible.

Block-level inheritance	Process change	Break page-level, then selective sync using Merge mode
Country fallback / virtual sites	Process change	Roll out from region directly — no custom config service
"Other" node catch-all	Process change	Same as country fallback
Recursive rollout across all levels	OOTB — pending product	Comes with multi-level inheritance
Path and link rewriting during rollout	OOTB — pending product	Comes with multi-level inheritance
Activation cascade	OOTB	Publishing cascades to all satellite levels
Custom relocation with redirect	Customisation required	DA plugin/app. Manual redirect fallback available.
MSM visualisation	Basic OOTB / Enhanced customisation	Enhanced view via DA plugin/app

Experience Fragments

Overview

This document defines how Experience Fragments (XF) are addressed in DA for TFS. It covers the fundamental shift in how fragments work in DA, how the AEM variation model is replaced by the MSM hierarchy, and how each functional XF requirement is met.

1. What Changes — AEM Sites vs DA

In AEM, Experience Fragments are a distinct content type — separate from pages, with their own authoring model, their own folder structure, and their own variation mechanism. Pages and Experience Fragments are two different things that are managed differently.

In DA, this distinction does not exist. **Fragments are regular documents.** They live in the same content hierarchy as pages, follow the same MSM inheritance rules as pages, and are authored the same way as any other content. There is no separate fragment type, no separate authoring model, and no separate resolution logic.

2. How Variations Work in AEM vs DA

AEM — Named Variations Under One Parent

In AEM, a single Experience Fragment holds multiple named variations under it. Authors create a variation per region, country, or language — each named according to the context it serves. A custom resolver then reads the page context (country, language, region) at render time and selects the best-matching variation to display.

A typical AEM XF structure looks like this:

```
1 /content/experience-fragments/tfsite/
2   └─ global-banner/
3       └─ master                ← default fallback
4       └─ en-US                ← US English variation
5       └─ en-GB                ← UK English variation
6       └─ fr-FR                ← French variation
7       └─ de-DE                ← German variation
8
```

All variations live under one parent. The resolver picks the right one at runtime based on the visitor's context. If no match is found, master is served as the fallback.

DA — Each Fragment Is Its Own Document in the Hierarchy

In DA, there are no named variations. Each fragment is a standalone document placed at the appropriate level in the MSM content hierarchy — exactly where that content belongs. The hierarchy itself replaces the variation model.

The same global banner in DA would look like this:

1	global/en/fragments/global-banner	← global level (replaces master)
2	north-america/en/fragments/global-banner	← North America level
3	en-us/fragments/global-banner	← US English level
4	europa/en/fragments/global-banner	← Europe level
5	en-gb/fragments/global-banner	← UK English level
6		

There are no named variations. There is no parent holding multiple children. Each fragment document is authored at its own level and inherits from its parent in the hierarchy if no local version exists.

3. How Fallback Works — No Custom Resolver Needed

AEM — Custom Best-Match Resolver

In AEM, when a page includes an XF, a custom resolver evaluates the page context and selects the best-matching variation using a priority order:

1. Country + Language match (e.g. en-US)
2. Language only
3. Region
4. Master (final fallback)

This resolver is a custom-built service that TFS maintains. It exists because AEM's MSM does not cover Experience Fragments — the variation model was built to simulate what MSM does for pages.

DA — MSM Hierarchy Handles Fallback Natively

In DA, the custom resolver is not needed. When a page needs a fragment, DA looks for it at the page's own level in the MSM hierarchy. If no fragment exists at that level, it cascades up to the parent level, then the grandparent, until it finds one.

For a US English page needing a banner fragment:

1	Look for fragment at US English level
2	→ Not found → look at North America level
3	→ Not found → look at Global English level
4	→ Found → serve this fragment

This is the same inheritance mechanism that governs page content. No custom code, no resolver logic, no variation matching — the hierarchy does the work.

When Fallback Happens

An important distinction: fallback in DA does not happen at visitor runtime. It happens at preview and publish time. By the time a visitor loads a page, the correct fragment is already resolved and available on the CDN. There is no per-visitor resolution happening in the browser.

4. Functional Use Cases — How Each Is Addressed

Use Case 1 — Best-Match XF Resolution (Standard)

What TFS does today: A custom resolver selects the best-matching XF variation for a page based on the page's country and language context. Resolution priority runs from Country + Language match down to master fallback. Variations are manually created and named per convention.

How it works in DA: The entire resolver and the variation model are replaced by the MSM hierarchy. The correct fragment for a given market is the fragment document that exists at that market's level in the hierarchy. If no local fragment exists, the hierarchy cascades up to the parent level automatically.

Authors create fragment documents at the levels where localization is needed — not as named variations under a parent, but as standalone documents in the hierarchy. Where no local version is authored, the parent level content flows down without any additional configuration.

Status: OOTB — MSM hierarchy handles this natively. Custom resolver not required.

Use Case 2 — Best-Match XF Resolution (Division — Forms)

What TFS does today: Form Experience Fragments use an additional dimension — division (e.g. CMD, GCMS, Eloqua-based) — on top of country and language matching. The form's division property selects from a division-specific sub-folder of XF variations, and the resolver then applies country and language matching within that scope.

How it works in DA: The division scoping logic, like the standard best-match resolver, exists because AEM MSM does not apply to Experience Fragments. In DA, fragments follow the same MSM hierarchy as pages. Division-specific form content is placed at the appropriate level in the

hierarchy — there is no need for a separate division dimension or a custom resolver to navigate it.

The MSM hierarchy handles both the market-level resolution and the form content scoping natively.

Status: OOTB — MSM hierarchy handles this natively. Division resolver not required.

5. Summary

Capability	AEM Approach	DA Approach	Status
Fragment content type	Separate content type from pages	Regular document in content hierarchy	Simplified — no separate type
Variation model	Named variations under one parent XF	Individual fragment documents at each hierarchy level	Replaced by MSM hierarchy
Best-match resolver (Standard)	Custom service — country + language priority matching	MSM hierarchy cascades up to parent if no local fragment	OOTB — resolver not required
Best-match resolver (Division/Forms)	Custom service — division scoping + country/language matching	MSM hierarchy handles natively	OOTB — resolver not required
Fallback	Master variation served if no match found	Parent-level fragment served at delivery time	OOTB — same as page fallback